Lecture 10 · Mon 11/2/2026

CS160 Compilers — Fall 2026

Semantic Analysis II: What Programs Mean.

Yu Feng

The judgment: G, E, S ⊢ e : v, S', R'.

manual §6.4, unchanged

$$G, E, S \vdash e : v, S', R'$$

G
the **global environment**: where top-level names live

E
the **local environment**: the parameters and locals of the current call

S
the **store**: what every memory location holds, plus the output printed so far

e
the expression or statement being evaluated

v
the value produced

S'
the store *afterwards*

R'
the value a `return` is carrying out, or `None`

⊢
left of it: what you start with; right of it: what you end with

E(x)
the location that `x` names — in `E` if `x` is local, in `G` otherwise.

Wednesday's `O, R ⊢ e : T` said which programs are ChocoPy. This judgment says what one *does*: *with globals G, locals E and store S, e evaluates to v, leaving store S' and return value R'.*

Big-step: one judgment goes from an expression straight to its final value. The store goes in and comes back out — that is how a judgment can say what a program *changed*, and in what order.

Rules with no premises: constants and variables.

$$\frac{}{G, E, S \vdash n : n, S, \text{None}}\ \text{E-Const} \qquad \frac{}{G, E, S \vdash x : S(E(x)), S, \text{None}}\ \text{E-Var}$$

A rule is **premises** over a line over a **conclusion**; the name at the right is a label, `E-` for expressions, `S-` for statements.

A constant means itself. A variable takes two steps: `E(x)` is the location, `S(E(x))` is what it holds. The same `S` comes out: these two rules cannot print, write or fail, and the rule says so.

Rules with premises: + and ⋆.

rule

$$\frac{G, E, S \vdash e_1 : v_1, S_1, \text{None} \qquad G, E, S_1 \vdash e_2 : v_2, S_2, \text{None}}{G, E, S \vdash e_1 + e_2 : \text{wrap}(v_1 + v_2), S_2, \text{None}}\ \text{E-Add}$$

$$\frac{G, E, S \vdash e_1 : v_1, S_1, \text{None} \qquad G, E, S_1 \vdash e_2 : v_2, S_2, \text{None}}{G, E, S \vdash e_1 * e_2 : \text{wrap}(v_1 \times v_2), S_2, \text{None}}\ \text{E-Mul}$$

interp10.py

```
case ast.BinOp(left=l, op=op, right=r):
    a = ev(l, env)
    b = ev(r, env)
    return wrap(BIN[type(op)](a, b))
```

Subscripts are names: e_1 the first operand, v_1 its value, S_1 the store it left behind.

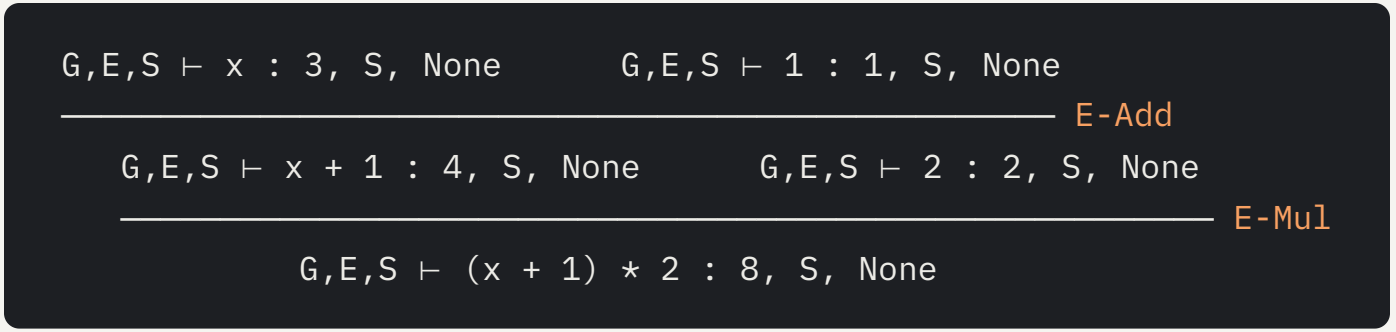
Premises are the recursive calls; the conclusion is the `return`.

Follow the stores: the first premise turns S into S_1; the second *starts from* S_1. The rule has said which operand goes first — slide 12.

`wrap`: ChocoPy's `int` is 32-bit (manual §2.5.1); the reference interpreter wraps past the range, and so must your compiler.

A derivation: (x + 1) * 2.

E = {x ↦ l}, S = {l ↦ 3}



↦

maps to: E = {x ↦ l} says x names location l; S = {l ↦ 3} says l holds 3

derivation

rules stacked: each bar one rule, everything above it a premise settled by another rule, the leaves rules with nothing above them.

Nothing here prints or writes, so every premise hands back the same S. This tree is the proof that the expression means 8; the interpreter's call stack builds exactly it.

Statements: `while`.

rule

```
G, E, S ⊢ c : False, S₁, None
──────────────────────────────────── S-WhileFalse
G, E, S ⊢ while c: s : _, S₁, None

G, E, S  ⊢ c : True, S₁, None
G, E, S₁ ⊢ s : _, S₂, None
G, E, S₂ ⊢ while c: s : _, S₃, R'
──────────────────────────────────── S-WhileTrue
G, E, S ⊢ while c: s : _, S₃, R'
```

interp10.py

```
case ast.While(test=c, body=b):
    while ev(c, env):
        for st in b: run(st, env)
```

`_` is a value nobody looks at: a statement's work is the store it hands back. Even `S-WhileFalse` returns S_1, because testing the condition may print or write.

Two rules, one `case`; Python's `while` picks the rule. The recursion in `S-WhileTrue` *is* the loop, and the store threads through it in the order the body runs.

Function calls.

rule

$$\frac{G, E, S \vdash e_1 : v_1, S_1, \text{None} \quad \dots \quad G, E, S_{n-1} \vdash e_n : v_n, S_n, \text{None} \qquad l_1 \dots l_n \text{ fresh} \qquad S_a = S_n[l_1 \mapsto v_1 \dots l_n \mapsto v_n] \qquad G, \{x_1 \mapsto l_1 \dots x_n \mapsto l_n\}, S_a \vdash \text{body of } f : _, S', v}{G, E, S \vdash f(e_1 \dots e_n) : v, S', \text{None}}\ \text{E-Call}$$

interp10.py

```
case ast.Call(func=ast.Name(id=f), args=args):
    vals = [ev(a, env) for a in args]                  # left to right
    local = {p.arg: v for p, v in zip(FUNS[f].args.args, vals)}
    try:
        for st in FUNS[f].body: run(st, local)
    except Return as r:
        return r.v
```

`Return`

an exception class of our own, not an error: `run` raises it at a `return`, `E-Call` catches it — the body's `R'` becomes the call's value.

A *fresh* environment for the body, built from the parameters only: the callee cannot see your locals. The *store* is not fresh — it threads through, so a callee's print is visible to everything after the call. Lecture 8's frame, as a rule.

The interpreter is the rules.

One `case` per rule (or per pair sharing a shape). Premises are recursive calls in the order the stores thread them; the conclusion is the `return`; the memory the interpreter mutates is S.

When the manual says ‘ChocoPy does X’: the rule says it, the interpreter does it, your compiler must match it.

Rules used this way are an **operational semantics**: a language defined by how it runs.

Well-typed programs do not get stuck.

stuck

a point where *no rule applies*: the program has no meaning there. `print(y)` with `y` never declared — no `E(y)`, so no location, so no rule

soundness

if the checker accepts a program, its evaluation never gets stuck.

Not *never fails* — *never runs out of rules*. Dividing by zero is not stuck: it has a rule, next slide.

Runtime errors are rules.

$$\frac{G, E, S \vdash e_1 : v_1, S_1, \text{None} \quad G, E, S_1 \vdash e_2 : v_2, S_2, \text{None} \quad v_2 \neq 0}{G, E, S \vdash e_1 \,//\, e_2 : v_1 \div v_2, S_2, \text{None}}\ \text{E-Div}$$

$$\frac{G, E, S \vdash e_1 : v_1, S_1, \text{None} \quad G, E, S_1 \vdash e_2 : 0, S_2, \text{None}}{G, E, S \vdash e_1 \,//\, e_2 : \textbf{error}, S_2, \text{None}}\ \text{E-DivZero}$$

`error`

an outcome the rules *name*: evaluation stops, exit status 1, and S_2 is the store it stopped in — everything printed before is out, nothing after ever is.

The manual lists exactly the runtime errors an implementation must produce (§6): bad argument to `print`/`len`, division by zero, index out of bounds, operation on `None`, out of memory. Each has a rule; nothing else may go wrong.

Undefined behavior: when there is no rule.

	C	ChocoPy, by rule
`x // 0`	undefined	runtime error, exit 1 — E-DivZero
`a[n] past the end`	undefined	runtime error — Wednesday's rule
`len(None)`	undefined (null dereference)	runtime error — Wednesday's rule
`signed overflow`	undefined	the manual fixes the range and stops (§2.5.1); the reference interpreter wraps, and that binds you

undefined behavior

no rule applies, so the language promises nothing: any output is consistent with the rules, and the compiler may assume it never happens.

Lecture 1's `average.c` read past its array and printed 25 at `-O0` and 0 at `-O2` (verified). Both are correct: there is no rule to break. In ChocoPy the same read has a rule.

Evaluation order is written in the rules.

$$\frac{\textcircled{1}\ G, E, S \vdash e_1 : v_1, S_1, \text{None} \qquad \textcircled{2}\ G, E, S_1 \vdash e_2 : v_2, S_2, \text{None}}{G, E, S \vdash e_1 + e_2 : \text{wrap}(v_1 + v_2), S_2, \text{None}}\ \text{E-Add}$$

$$S \xrightarrow{\textcircled{1}} S_1 \xrightarrow{\textcircled{2}} S_2$$

Evaluation order: when a rule has two premises, which is settled first?

The rule answers by itself: ② starts from S_1, the store ① produced, so ② cannot be settled until ① has been. Swap the two and you have written a *different rule*.

C leaves this unspecified. ChocoPy fixes it — left to right — in the subscripts on the stores.

A program whose output depends on it.

order.py · verified

```
def f() -> int:
    print(1)
    return 10

def g() -> int:
    print(2)
    return 20

print(f() + g())
```

pre-rendered · verified

interpreter	compiled	compiled, the two compile_expr lines in BinOp swapped
1	1	**2**
2	2	**1**
30	30	**30**

2 1 30 is an output no derivation produces. Lecture 4's oracle catches the swapped compiler in one run; the rule is what says it was wrong.

The IR shows the choice.

order.ll · verified

```
%t1 = call i32 @f()%t2 = call i32 @g()
%t3 = add i32 %t1, %t2
call void @print_int(i32 %t3)
```

add is commutative. The two `call`s above it are not. The compiler had to pick an order; the rule told it which.

Side effects: is e * 0 → 0 valid?

times0.py · verified · prints 7 then 0

```
def tick() -> int:
    print(7)
    return 3

print(tick() * 0)
```

side effect

anything code does besides producing a value: printing, stopping with an error, changing something the rest of the program can see — in the rules, $S_1 \neq S$

observationally equivalent

two programs nobody can tell apart from outside: same prints, same errors, same order.

E-Mul settles `tick()` first; its print is the change $S \to S_1$. Rewriting to `0` drops the 7: different output, invalid. *Valid* means observationally equivalent — the standard every optimization in weeks 8–9 is held to.

When the rewrite is valid.

`x * 0 → 0` when `x` is a variable: `E-Var` hands back the store it was given, so there is nothing to observe.

`f() + g() → g() + f()` only if neither prints, errors or writes a global — only if both premises return the store unchanged — and the compiler cannot know that without looking inside both.

Exercise #1: write the derivation for `inc(x) * x`.

deriv.py · verified · prints 6

```
def inc(a: int) -> int:
    return a + 1

x: int = 2
print(inc(x) * x)
```

E = {x ↦ l}, S = {l ↦ 2}. Derive `G, E, S ⊢ inc(x) * x : 6, S, None`, labelling every rule.

Exercise #2: is `e * 0 → 0` valid? Three cases.

(a) e is a variable `x`.

(b) e is `f()` where `f` prints.

(c) e is `f()` whose body is only `return 3`.

Valid or not, one sentence each, in terms of what the rules make observable.

Wednesday; PA4.

Wednesday: where a list lives, how `xs[i]` is compiled with its bounds check, and the rules for index-out-of-bounds and None.

PA4 (out today, due Fri 11/13): lists and strings. The interpreter you are given already has your handout's rules (page 1); the hidden tests were generated from it.